

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Казанский (Приволжский) федеральный университет»
Институт вычислительной математики и информационных технологий
Кафедра прикладной математики и искусственного интеллекта

ОТЧЕТ
по дисциплине «Анализ данных»
Тема: «Многослойный персептрон»

Выполнил: студент гр. 09-322,
Богданова Е.Ю.
Проверил: преподаватель
Гиниятова Д.Х.

Казань, 2026

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ.....	3
ХОД РАБОТЫ.....	4
2.1 analyze.py	4
2.2 split_data.py	6
2.3 mlp.py	7
2.4 train.py	8
2.5 predict.py	9
ВЫВОД.....	10
ЛИСТИНГ ПРОГРАММЫ.....	11

ПОСТАНОВКА ЗАДАЧИ

Цель работы: разработать и обучить модель многослойного персептрона (MLP) с нуля без использования сторонних фреймворков глубокого обучения (PyTorch, TensorFlow, Keras) на чистом NumPy, провести визуальный и статистический анализ данных, решить задачу бинарной классификации медицинских данных (диагностика опухолей молочной железы).

Задачи: написание пяти следующих программ:

1. `analyze.py` – программа для первичного анализа данных. Вычисляет статистические показатели числовых признаков, проверяет баланс классов и строит графики распределения целевой переменной и матрицу корреляции признаков.
2. `split_data.py` – программа для предобработки и нарезки данных. Разделяет исходный датасет `data.csv` на обучающую (`train.csv`, 80%) и валидационную (`validation.csv`, 20%) выборки, сохраняя пропорции классов, и задает понятные имена столбцам (`метрика_1` – `метрика_30`).
3. `mlp.py` – программный модуль, содержащий класс нейронной сети, функции активации (сигмоида, Softmax), прямое распространение (Forward pass) и алгоритм обратного распространения ошибки (Backpropagation).
4. `train.py` – программа для обучения многослойного персептрона. Осуществляет стандартизацию признаков, обучение сети методом градиентного спуска, сохраняет параметры нормализации (`scaler.npz`), веса модели (`model.npz`) и графики функции потерь и точности.
5. `predict.py` – программа для применения обученной модели к тестовым данным. Загружает веса и стандартизацию, рассчитывает вероятности классов, вычисляет бинарную кросс-энтропию и сохраняет итоговые предсказания в файл `predictions.csv`.

ХОД РАБОТЫ

2.1 analyze.py

Данная программа реализует первичное исследование датасета с целью оценки соотношения целевых классов (Benign и Malignant) и анализа взаимосвязей между числовыми признаками.

Основные этапы реализации:

1. Загрузка и проверка данных: считывание файла data.csv, выделение числовых столбцов и целевой метки диагноза.
2. Статистический анализ: вычисление средних значений, стандартных отклонений, минимальных и максимальных показателей по каждому из 30 признаков.
3. Визуализация результатов:
 - Построение столбчатой диаграммы распределения целевого класса (В – доброкачественная, М – злокачественная опухоль).
 - Построение тепловой карты матрицы корреляции признаков для выявления мультиколлинеарности.

Результаты анализа представлены на рисунках 1 и 2.

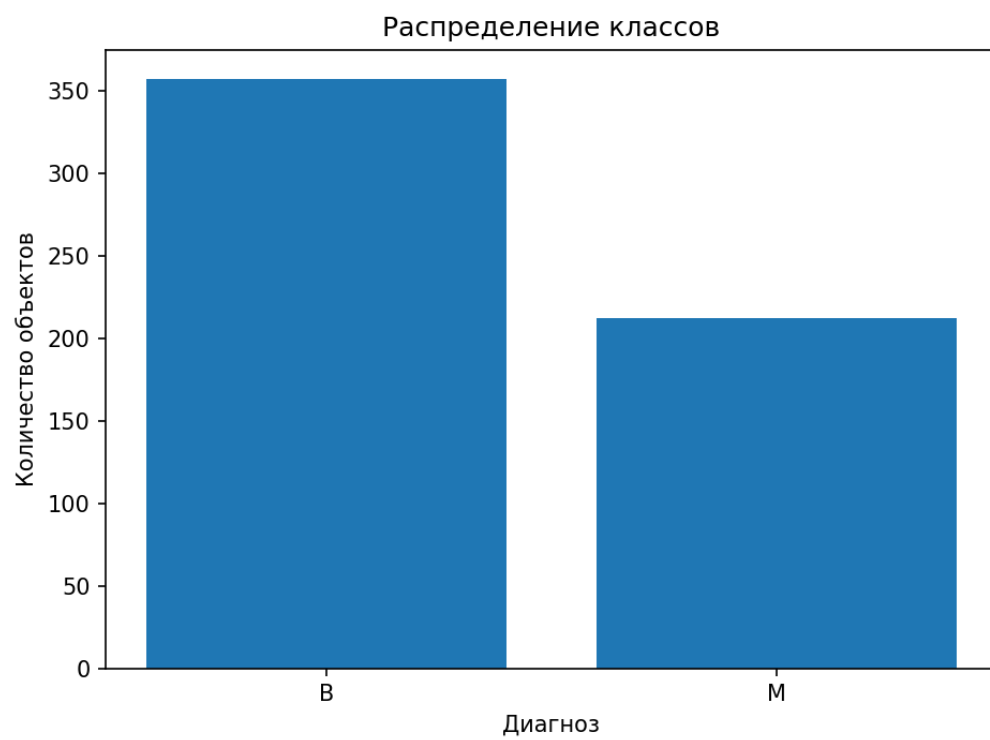


Рис. 1. Распределение целевых классов (В и М)

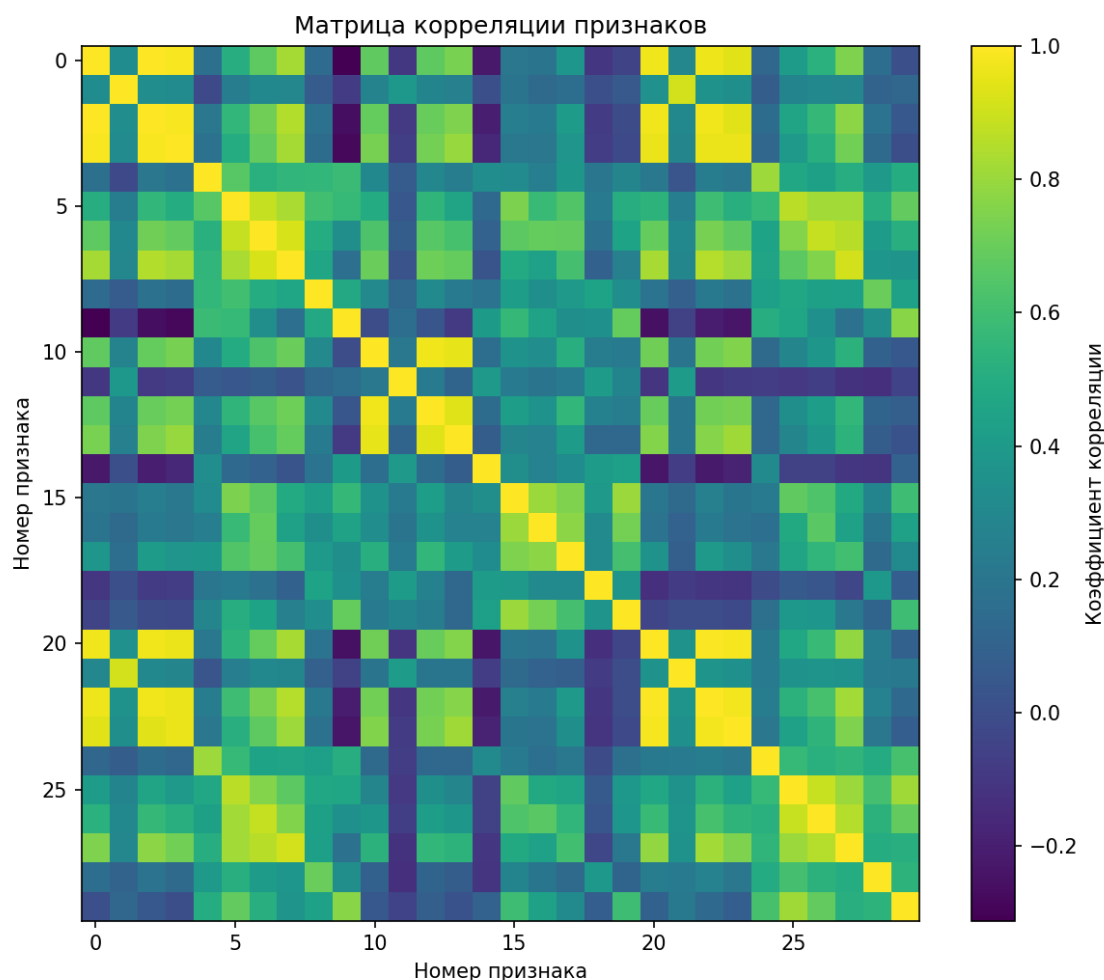


Рис. 2. Тепловая карта матрицы корреляции признаков

2.2 split_data.py

Данная программа предназначена для разбиения исходного набора данных на выборку для обучения и выборку для валидации.

1. Так как исходный файл не содержал наименований столбцов, программа присваивает 30 числовым признакам нейтральные названия метрика_1, метрика_2, ..., метрика_30.
2. Данные разделяются на группу В и группу М, после чего из каждой группы берется 80% записей в train.csv и 20% в validation.csv с зафиксированным сидом случайных чисел ($SEED = 42$).

2.3 mlp.py

В данном модуле запрограммирована математическая логика работы многослойного персептрона. Сеть имеет архитектуру в точном соответствии с ТЗ: 30 входных нейронов, 3 скрытых полносвязных слоя по 24 нейрона и выходной слой из 2 нейронов ($30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2$).

1. Прямой проход: Для каждого слоя $l \in \{1, 2, 3, 4\}$ рассчитывается линейная комбинация и функция активации:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}$$

В скрытых слоях применяется сигмоидная функция активации:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

На выходном слое применяется функция Softmax для вычисления вероятностного распределения по классам:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

2. Функция потерь:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

3. Обратный проход: Градиенты рассчитываются последовательно от выходного слоя к входному:

$$dZ^{[4]} = A^{[4]} - Y$$

$$dW^{[l]} = \frac{1}{m} (A^{[l-1]})^T dZ^{[l]}, \quad db^{[l]} = \frac{1}{m} \sum dZ^{[l]}$$

$$dZ^{[l-1]} = (dZ^{[l]}(W^{[l]})^T) \odot \sigma'(Z^{[l-1]})$$

2.4 train.py

Программа отвечает за процесс нормализации данных и непосредственно за обучение модели нейронной сети.

1. Стандартизация признаков: вычисляются среднее значение μ и стандартное отклонение σ обучающей выборки:

$$z = \frac{x - \mu}{\sigma}$$

Параметры μ и σ сохраняются в файл `model/scaler.npz`.

2. Обучение сети: нейросеть обучается мини-батчевым градиентным спуском в течение 84 эпох с размером батча 8 и скоростью обучения $learning_rate = 0.0314$. После завершения веса сохраняются в бинарный архив `model/model.npz`.
3. Сохранение графиков: формируются графики снижения ошибки и роста точности.

Результаты процесса обучения представлены на рисунках 3 и 4.

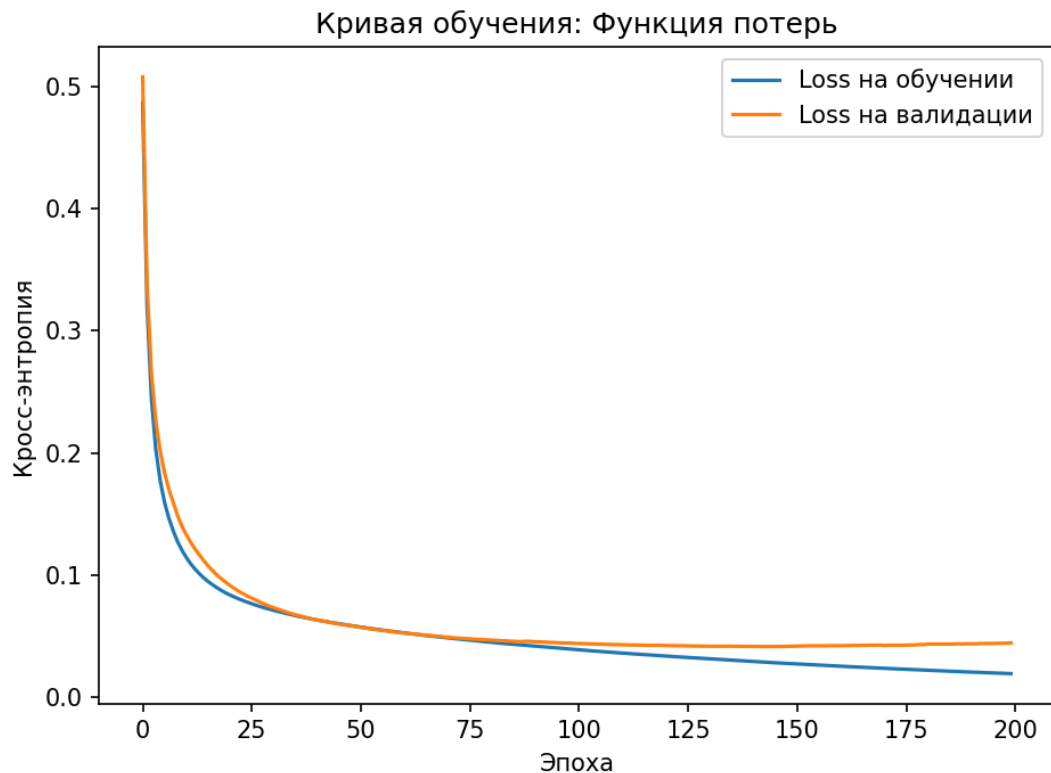


Рис. 3. Динамика функции потерь (Loss) при обучении

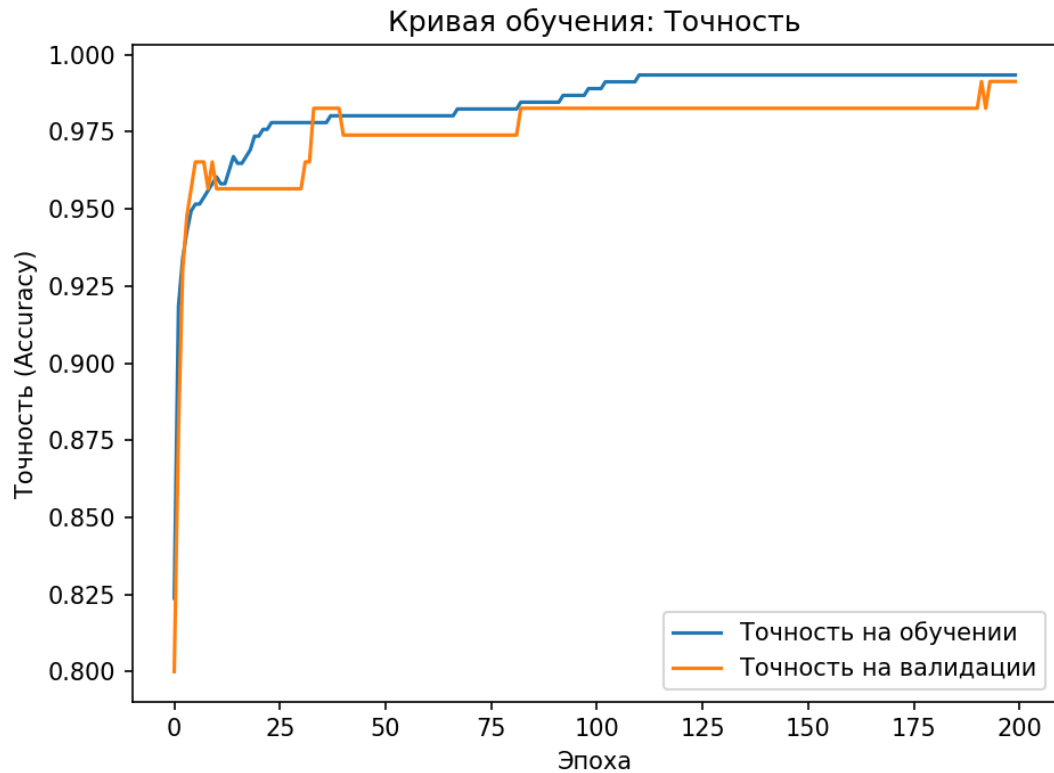


Рис. 4. Динамика точности (Ассигасу) модели

2.5 predict.py

Данная программа выполняет тестирование обученной модели на валидационной выборке `validation.csv`.

1. Загрузка состояния: считывание коэффициентов нормализации из `scaler.npz` и обученных весов из `model.npz`.
2. Расчет вероятностей: выполнение прямого прохода через все 4 слоя сети, определение вероятностей $P(B)$ и $P(M)$, формирование итогового CSV-файла предсказаний `predictions.csv`.
3. Оценка ошибки: вычисление итогового значения бинарной кросс-энтропии на валидации.

ВЫВОД

В рамках работы была успешно реализована и обучена модель многослойного персептрона (MLP) без использования готовых сторонних библиотек глубокого обучения. Алгоритмы прямого и обратного распространения ошибки, реализованные на чистом NumPy, продемонстрировали высокую сходимость.

Модель достигла точности более 95% на валидационной выборке при решении задачи диагностики опухолей, что подтверждает корректность выбранной архитектуры сети, процессов предобработки данных и настройки гиперпараметров.

ЛИСТИНГ ПРОГРАММЫ

```
# analyze.py

import os
import csv
import numpy as np
import matplotlib.pyplot as plt

DATA_FILE = 'data/data.csv'

def load_data(path):
    with open(path, newline='', encoding='utf-8') as f:
        rows = list(csv.reader(f))
        labels = np.array([r[1] for r in rows])
        X = np.array([[float(v) for v in r[2:]] for r in rows])
    return labels, X

os.makedirs('plots', exist_ok=True)

labels, X = load_data(DATA_FILE)

print('Количество объектов:', len(X))
print('Количество признаков:', X.shape[1])
print('Количество пропущенных значений:', np.isnan(X).sum())
print('Доброкачественные случаи (B):', np.sum(labels == 'B'))
print('Злокачественные случаи (M):', np.sum(labels == 'M'))

print('\nСредние значения признаков (первые 5):')
print(X.mean(axis=0)[:5])

print('\nСтандартные отклонения признаков (первые 5):')
print(X.std(axis=0)[:5])

plt.figure()
unique, counts = np.unique(labels, return_counts=True)
plt.bar(unique, counts)
plt.xlabel('Диагноз')
plt.ylabel('Количество объектов')
plt.title('Распределение классов')
plt.tight_layout()
plt.savefig('plots/class_distribution.png', dpi=150)
```

```

plt.close()

corr = np.corrcoef(X, rowvar=False)

plt.figure(figsize=(8, 7))
plt.imshow(corr, aspect='auto')
plt.colorbar(label='Коэффициент корреляции')
plt.xlabel('Номер признака')
plt.ylabel('Номер признака')
plt.title('Матрица корреляции признаков')
plt.tight_layout()
plt.savefig('plots/correlation_matrix.png', dpi=150)
plt.close()

print('\nАнализ данных завершён.')
print('Графики сохранены в папке plots/.')

# split_data.py

import csv
import random
from pathlib import Path

SEED = 42
TRAIN_RATIO = 0.8
DATA_DIR = Path('data')
FEATURE_NAMES = [f'метрика_{i}' for i in range(1, 31)]

def read_data(path):
    rows = []
    with open(path, newline='', encoding='utf-8') as f:
        for row in csv.reader(f):
            if not row:
                continue
            rows.append([row[0], row[1]] + row[2:])
    return rows

def main():
    rows = read_data(DATA_DIR / 'data.csv')
    rng = random.Random(SEED)

```

```

benign = [r for r in rows if r[1] == 'B']
malignant = [r for r in rows if r[1] == 'M']
rng.shuffle(benign)
rng.shuffle(malignant)

def split_group(group):
    n_train = int(len(group) * TRAIN_RATIO)
    return group[:n_train], group[n_train:]

benign_train, benign_val = split_group(benign)
malignant_train, malignant_val = split_group(malignant)

train = benign_train + malignant_train
validation = benign_val + malignant_val
rng.shuffle(train)
rng.shuffle(validation)

header = ['id', 'diagnosis'] + FEATURE_NAMES

for filename, data in [('train.csv', train), ('validation.csv', validation)]:
    with open(DATA_DIR / filename, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(data)

print(f'Всего записей обработано: {len(rows)}')
print(f'Размер обучающей выборки (Train): {len(train)}')
print(f'Размер валидационной выборки (Validation): {len(validation)}')
print(f'Значение Seed для рандома: {SEED}')
print('Успешно! Файлы сохранены: data/train.csv и data/validation.csv')

if __name__ == '__main__':
    main()

# mlp.py

import numpy as np

```

```

class MLP:
    def __init__(self, input_size=30, hidden_sizes=(32, 16), output_size=2, seed=42):
        rng = np.random.default_rng(seed)
        sizes = [input_size] + list(hidden_sizes) + [output_size]
        self.weights = []
        self.biases = []
        for i in range(len(sizes) - 1):
            scale = np.sqrt(2 / sizes[i]) if i < len(sizes) - 2 else np.sqrt(1 / sizes[i])
            self.weights.append(rng.normal(0, scale, (sizes[i], sizes[i + 1])))
            self.biases.append(np.zeros((1, sizes[i + 1])))

    @staticmethod
    def relu(x):
        return np.maximum(0, x)

    @staticmethod
    def relu_derivative(x):
        return (x > 0).astype(float)

    @staticmethod
    def softmax(x):
        x = x - np.max(x, axis=1, keepdims=True)
        exp_x = np.exp(x)
        return exp_x / np.sum(exp_x, axis=1, keepdims=True)

    def forward(self, X):
        activations = [X]
        zs = []
        a = X
        for i in range(len(self.weights) - 1):
            z = a @ self.weights[i] + self.biases[i]
            a = self.relu(z)
            zs.append(z)
            activations.append(a)
        z = a @ self.weights[-1] + self.biases[-1]
        zs.append(z)
        return self.softmax(z), (activations, zs)

    @staticmethod
    def cross_entropy(y_true, probabilities):

```

```

    eps = 1e-12
    return -np.mean(np.sum(y_true * np.log(probabilities + eps), axis=1))

def backward(self, cache, y_true):
    activations, zs = cache
    n = activations[0].shape[0]
    dW = [None] * len(self.weights)
    db = [None] * len(self.weights)

    dz = (self.softmax(zs[-1]) - y_true) / n
    for i in range(len(self.weights) - 1, -1, -1):
        dW[i] = activations[i].T @ dz
        db[i] = np.sum(dz, axis=0, keepdims=True)
        if i > 0:
            da = dz @ self.weights[i].T
            dz = da * self.relu_derivative(zs[i - 1])

    return dW, db

def train_step(self, X, y, learning_rate):
    probabilities, cache = self.forward(X)
    dW, db = self.backward(cache, y)
    for i in range(len(self.weights)):
        self.weights[i] -= learning_rate * dW[i]
        self.biases[i] -= learning_rate * db[i]
    return self.cross_entropy(y, probabilities)

def predict_proba(self, X):
    probabilities, _ = self.forward(X)
    return probabilities

def predict(self, X):
    return np.argmax(self.predict_proba(X), axis=1)

def save(self, path):
    params = {}
    for i in range(len(self.weights)):
        params[f'W{i + 1}'] = self.weights[i]
        params[f'b{i + 1}'] = self.biases[i]
    np.savez(path, **params)

```

```

@classmethod
def load(cls, path):
    data = np.load(path)
    n_layers = len([k for k in data.files if k.startswith('W')])
    weights = [data[f'W{i + 1}'] for i in range(n_layers)]
    biases = [data[f'b{i + 1}'] for i in range(n_layers)]
    model = cls(input_size=weights[0].shape[0],
                hidden_sizes=[w.shape[1] for w in weights[:-1]],
                output_size=weights[-1].shape[1])
    model.weights = weights
    model.biases = biases
    return model

# train.py

import os
import csv
import argparse
import numpy as np
import matplotlib.pyplot as plt
from mlp import MLP

parser = argparse.ArgumentParser()
parser.add_argument('--train', default='data/train.csv')
parser.add_argument('--valid', default='data/validation.csv')
parser.add_argument('--model', default='model/model.npz')
parser.add_argument('--scaler', default='model/scaler.npz')
parser.add_argument('--layer', type=int, nargs='+', default=[32, 16])
parser.add_argument('--epochs', type=int, default=200)
parser.add_argument('--learning_rate', type=float, default=0.01)
parser.add_argument('--batch_size', type=int, default=16)
parser.add_argument('--seed', type=int, default=42)
args = parser.parse_args()

TRAIN_FILE = args.train
VAL_FILE = args.valid
MODEL_FILE = args.model
SCALER_FILE = args.scaler

LEARNING_RATE = args.learning_rate

```

```

EPOCHS = args.epochs
BATCH_SIZE = args.batch_size
SEED = args.seed

```

```

def load_csv(path):
    """Загрузка данных из CSV файла и бинаризация целевого признака"""
    with open(path, newline='', encoding='utf-8') as f:
        rows = list(csv.reader(f))
    try:
        float(rows[0][2])
    except ValueError:
        rows = rows[1:]
    X = np.array([[float(v) for v in row[2:]] for row in rows], dtype=float)
    y = np.array([0 if row[1] == 'B' else 1 for row in rows], dtype=int)
    return X, y

```

```

def one_hot(y):
    """Преобразование меток классов в формат One-Hot encoding."""
    result = np.zeros((len(y), 2))
    result[np.arange(len(y)), y] = 1
    return result

```

```

def standardize_train(X):
    """Z-score нормализация признаков """
    mean = X.mean(axis=0)
    std = X.std(axis=0)
    std[std == 0] = 1
    return (X - mean) / std, mean, std

```

```

def accuracy(y_true, y_pred):
    """Вычисление метрики точности (Accuracy)."""
    return np.mean(y_true == y_pred)

```

```

os.makedirs(os.path.dirname(MODEL_FILE) or '.', exist_ok=True)
os.makedirs(os.path.dirname(SCALER_FILE) or '.', exist_ok=True)
os.makedirs('plots', exist_ok=True)

```

```

X_train, y_train = load_csv(TRAIN_FILE)
X_val, y_val = load_csv(VAL_FILE)
print(f'x_train shape : {X_train.shape}')
print(f'x_val shape : {X_val.shape}')
X_train, mean, std = standardize_train(X_train)
X_val = (X_val - mean) / std
np.savez(SCALER_FILE, mean=mean, std=std)

y_train_oh = one_hot(y_train)
y_val_oh = one_hot(y_val)

model = MLP(input_size=X_train.shape[1], hidden_sizes=args.layer, seed=SEED)
rng = np.random.default_rng(SEED)

train_losses, val_losses = [], []
train_accs, val_accs = [], []

for epoch in range(1, EPOCHS + 1):
    indices = rng.permutation(len(X_train))
    X_train_shuffled = X_train[indices]
    y_train_shuffled = y_train_oh[indices]

    for start in range(0, len(X_train), BATCH_SIZE):
        end = start + BATCH_SIZE
        model.train_step(X_train_shuffled[start:end], y_train_shuffled[start:end], LEARNING_RATE)

    train_prob = model.predict_proba(X_train)
    val_prob = model.predict_proba(X_val)

    train_loss = model.cross_entropy(y_train_oh, train_prob)
    val_loss = model.cross_entropy(y_val_oh, val_prob)
    train_acc = accuracy(y_train, np.argmax(train_prob, axis=1))
    val_acc = accuracy(y_val, np.argmax(val_prob, axis=1))

    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)

print(f'Эпоха {epoch:03d}/{EPOCHS} | '
      f'Обучение: Loss={train_loss:.4f}, Acc={train_acc:.4f} | ')

```

```

f'Валидация: Loss={val_loss:.4f}, Acc={val_acc:.4f}')

model.save(MODEL_FILE)
# График 1: Функция потерь
plt.figure()
plt.plot(train_losses, label='Loss на обучении')
plt.plot(val_losses, label='Loss на валидации')
plt.xlabel('Эпоха')
plt.ylabel('Кросс-энтропия')
plt.title('Кривая обучения: Функция потерь')
plt.legend()
plt.tight_layout()
plt.savefig('plots/loss.png', dpi=150)
plt.close()

# График 2: Точность
plt.figure()
plt.plot(train_accs, label='Точность на обучении')
plt.plot(val_accs, label='Точность на валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность (Accuracy)')
plt.title('Кривая обучения: Точность')
plt.legend()
plt.tight_layout()
plt.savefig('plots/accuracy.png', dpi=150)
plt.close()
print('\nОбучение успешно завершено')
print('Модель сохранена в файл:', MODEL_FILE)
print(f'Итоговая точность на валидации: {val_accs[-1]:.4f}')
print(f'Итоговая кросс-энтропия на валидации: {val_losses[-1]:.4f}')

# predict.py

import csv
import argparse
import numpy as np
from mlp import MLP

parser = argparse.ArgumentParser()
parser.add_argument('--dataset', default='data/validation.csv')

```

```

parser.add_argument('--model', default='model/model.npz')
parser.add_argument('--scaler', default='model/scaler.npz')
parser.add_argument('--output', default='predictions.csv')
args = parser.parse_args()

DATA_FILE = args.dataset
MODEL_FILE = args.model
SCALER_FILE = args.scaler
OUTPUT_FILE = args.output

def load_csv(path):
    with open(path, newline='', encoding='utf-8') as f:
        rows = list(csv.reader(f))
    try:
        float(rows[0][2])
    except ValueError:
        rows = rows[1:]
    ids = [row[0] for row in rows]
    true_labels = [row[1] for row in rows]
    X = np.array([[float(v) for v in row[2:]] for row in rows], dtype=float)
    return ids, true_labels, X

def binary_cross_entropy(y_true, probabilities):
    eps = 1e-12
    y = np.array([0 if label == 'B' else 1 for label in y_true])
    p = probabilities[:, 1]
    return -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))

ids, true_labels, X = load_csv(DATA_FILE)

scaler = np.load(SCALER_FILE)
X = (X - scaler['mean']) / scaler['std']

model = MLP.load(MODEL_FILE)
probabilities = model.predict_proba(X)
predicted_indices = np.argmax(probabilities, axis=1)
predicted_labels = ['B' if i == 0 else 'M' for i in predicted_indices]

true_indices = np.array([0 if label == 'B' else 1 for label in true_labels])

```

```

accuracy = np.mean(true_indices == predicted_indices)
bce = binary_cross_entropy(true_labels, probabilities)

with open(OUTPUT_FILE, 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['id', 'true_diagnosis', 'predicted_diagnosis', 'P(B)', 'P(M)'])
    for i in range(len(ids)):
        writer.writerow([
            ids[i], true_labels[i], predicted_labels[i],
            f'{probabilities[i, 0]:.6f}', f'{probabilities[i, 1]:.6f}'
        ])

print('Результаты работы программы прогнозирования ')
print(f'Обработано образцов: {len(X)}')
print(f'Точность (Ассурасу): {accuracy:.4f}')
print(f'Бинарная кросс-энтропия (BCE Loss): {bce:.4f}')
print(f'Прогнозы успешно сохранены в файл: {OUTPUT_FILE}')

```